

Brand Visibility Intelligence

Project report | 24 September 2026

A Python and SQLite analytics project with a six-tab Streamlit dashboard. Prepared against the supplied dashboard task brief and Details.txt.

Completion status

Application, CSV cleaning, SQL integration, 30 cleaning answers, 30 EDA answers, automated tests, clean dataset and dashboard screenshots are implemented. Live API collection and verification remain pending a configured SerpAPI key. The exact brief-named brand_dirty_dataset.csv was not supplied; the available local brand_visibility.csv was used for this run, pending confirmation that it is the intended input.

Source and coverage

Input: brand_visibility.csv found in Downloads, with 1,320 rows and seven original columns. Source currency is not stated. Search position and original price are absent, so visibility and discount findings cannot be computed from this CSV. No sample ranks or discount values were substituted.

Measure	Observed result
Input rows	1320
Clean rows	1226
Duplicate offers removed	94
Price values imputed	207
Prices capped	13
Observed rankings	0
Known discounts	0

Architecture

CSV + SerpAPI Google Shopping -> aligned DataFrames -> cleaning and derived metrics -> cleaned CSV and indexed SQLite products table -> parameterized SQL filters -> Streamlit tabs and reproducible reports. The main app starts with one command. A batch pipeline is provided for repeatable runs.

Cleaning and feature engineering

Column names and category labels are normalized. Numeric parsing accepts whole numeric strings with currency/grouping symbols and rejects invalid text. Price uses keyword medians then an overall median when missing; price_imputed records replacements. Missing ratings, reviews, ranks and original prices remain unknown. Records without a usable title or price are removed. Duplicates use keyword, cleaned title, platform and uncapped price. Prices are capped at the 99th percentile; price_before_cap preserves auditability.

Visibility is 100 / observed position. Discounts use valid observed original and selling prices before capping, and are not calculated for imputed selling prices. Price and rating categories use exact boundaries. Brand extraction is heuristic. Missing SQL values remain NULL and do not masquerade as zero or a fabricated rank.

Dashboard coverage

Overview: product count, average price/rating, reviews, distributions and platform share. Brand Insights: count, ratings and top-10 placements. Pricing: prices, rating/ranking relationships and discounts. Platform Analysis: count, price, ratings, rankings and brand mix. Visibility & Ranking: position, score, top-10 share and relationships. Product Explorer: search, sortable full table, top observed placements, charts and CSV download.

Sidebar controls cover brand, platform, price range, rating range, keyword and observed position. Missing ranking data disables the ranking filter and displays a clear explanation. A CSV/API loading panel supports merging only after common-currency confirmation. API calls occur only on explicit load, one page per keyword.

Findings from the available CSV

The following observations describe only this CSV and are not live market estimates. Listing share is not sales or market share.

HP has the most listings (157; 12.8% of this selection). Compare its assortment before expanding competing products.

Croma has the highest observed average rating (3.83/5). Review sample sizes and customer feedback before prioritizing that platform.

Search positions are absent. Collect ranked API results before making visibility recommendations.

Discount/ranking comparisons need observed prices and ranks in both discounted and non-discounted groups.

Results describe the supplied listings, not sales, market share, or causal drivers of ranking.

Interpretation limits

No rankings means no supported ranking/visibility conclusions. No original prices means no supported discount conclusions. Missing ratings/reviews are excluded from corresponding means. A pooled price cap may compress legitimate high-price categories. Correlations and top-10 comparisons are descriptive; they do not establish what causes ranking. Currency must be confirmed before combining CSV and API results.

Verification

Nine automated tests pass: existing cleaning behavior; missing-rank/discount preservation and SQLite round-trip; invalid values and category boundaries; API response normalization and installment exclusion; API error handling without credential exposure; parameterized SQL filtering; exported reports; and all six dashboard tabs with ranked and wholly unranked/unrated data. Both dashboard cases also exercise brand filtering and an empty product search. A browser check confirms the local CSV dashboard renders.

Live API limitation

The API adapter is implemented against the SerpAPI Google Shopping contract and verified with controlled responses. This is not a successful live API run. No SERPAPI_KEY was configured at delivery time. After setting it in local .env, use Load data or the batch command with --api. Raw extracted API rows are saved by the batch pipeline. Monthly installment listings are excluded to avoid comparing recurring payments with outright prices.

Run locally

```
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

Build a dataset

```
python pipeline.py --csv /path/to/products.csv
```

For combined data: add --api --keywords laptop phone. Confirm a single currency before running a combined batch. The app also offers the same workflow through Load data.

Submission files

deliverables/cleaned_products.csv; deliverables/brand_visibility.db; deliverables/screenshots/; output/pdf/brand_visibility_project_report.pdf; reports/cleaning_answers.md; reports/eda_answers.md; reports/cleaning_report.json; source code and tests. The report generator is scripts/build_project_report.py.

Remaining user inputs

Confirm that brand_visibility.csv is the intended assignment CSV, or provide brand_dirty_dataset.csv. Configure SERPAPI_KEY locally and confirm the CSV currency/target API market. Then run and verify the live combined-data pipeline and regenerate deliverables. Until those steps are completed, the assignment is not fully verified with live data.

References

User-supplied Brand Visibility Intelligence Dashboard (1).pdf and Details.txt. API reference:
<https://serpapi.com/google-shopping-api>. Repository:
<https://github.com/Santhoshkarmegham/brand-visibility-simple>